# Procesos de Desarrollo de Software

CLASE 10 – PATRONES DE DISEÑO

# Agenda de la clase

- Strategy

# Strategy

Es un patrón **de Comportamiento.**

**¿Qué hace?**

- Encapsula distintas formas (o algoritmos) de resolver el mismo problema en diferentes clases.
- Permite intercambiar en momento de ejecución la forma en que un tercero resuelve un problema.

Se **sugiere** su utilización **cuando**:

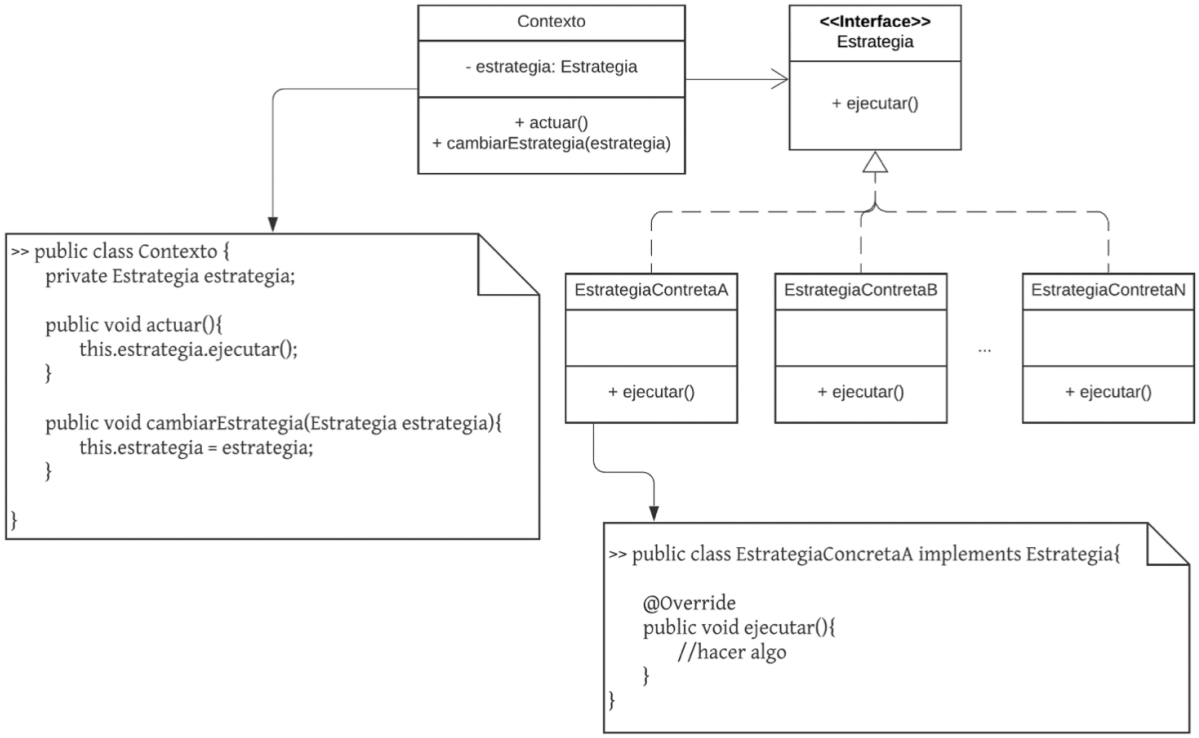
- Cuando varias reglas resuelven el mismo problema de maneras distintas.
- Cuando los condicionales crecen cada vez que aparece una nueva variante.

# Componentes

Componentes:

- **Interface (o clase abstracta) Strategy**: Define la firma del método que se utilizará como estrategia de resolución de algún problema.
- **Clases de estrategias concretas**: Clases que implementan la interface Strategy (o que heredan de ella, si ésta fuera clase abstracta), es decir, que tienen la implementación real del algoritmo.
- **Contexto**: Clase que tiene referencia a la interface/clase abstracta Strategy, cuyos objetos van a delegar la responsabilidad de resolución de algún problema en la estrategia. Estos objetos utilizarán de forma polimórfica a las estrategias concretas.

# Diagrama UML



# Ejemplo

- Un carrito debe calcular el costo de entrega.
- Envío normal y envío express usan reglas diferentes.
- El carrito necesita una sola operación: calcular(total).

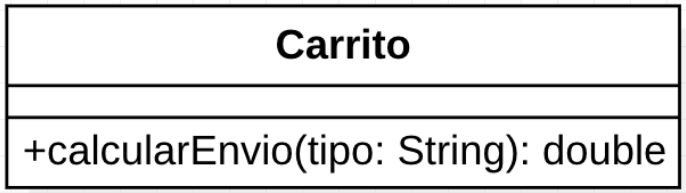
**Envío normal**

$1.500 fijos

**Envío express**

$3.000 fijos

# Solución: Sin Strategy

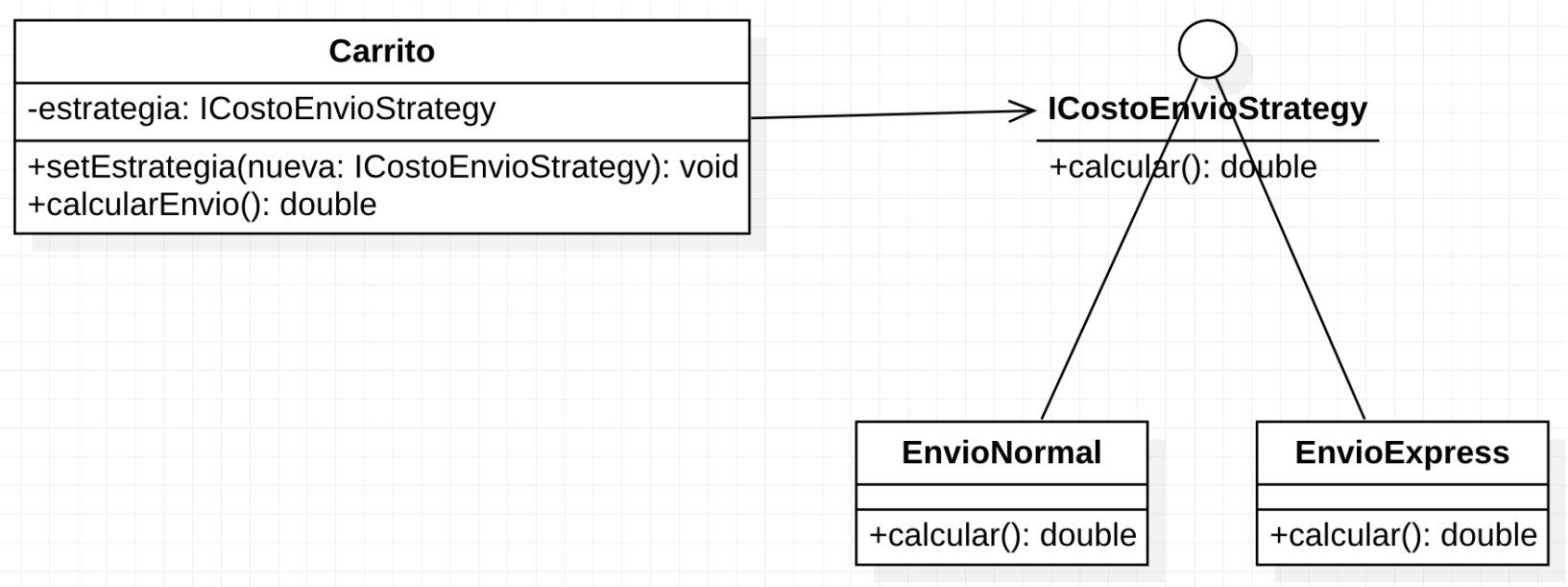



```java
public class Carrito {

    public double calcularEnvio(String tipo) {
        if (tipo.equals("NORMAL")) {
            return 1500;
        } else if (tipo.equals("EXPRESS")) {
            return 3000;
        }
        throw new IllegalArgumentException();
    }

}
```

# Solución: Con Strategy



# Solución: Con Strategy

```java
public class Carrito {
    private ICostoEnvioStrategy estrategia;

    public void setEstrategia(ICostoEnvioStrategy estrategia) {
        this.estrategia = estrategia;
    }

    public double calcularEnvio() {
        return estrategia.calcular();
    }
}
```

```java
public interface ICostoEnvioStrategy {
    double calcular();
}
```

```java
public class EnvioNormal implements ICostoEnvioStrategy {
    @Override
    public double calcular() {
        // Lógica para calcular el costo de envío normal
        return 1500; // Ejemplo de costo fijo
    }
}

public class EnvioExpress implements ICostoEnvioStrategy {
    @Override
    public double calcular() {
        // Lógica para calcular el costo de envío express
        return 3000; // Ejemplo de costo fijo
    }
}
```

# Práctica

Realicemos el ejercicio de aplicación de patrones "**Módulo de envío de notificaciones**".

# Magic Café ☕ ✨

- El proyecto calcula descuentos al confirmar un Pedido.
- La regla elegida puede ser: sin descuento, estudiante o hechizo.
- Strategy permite agregar nuevos descuentos sin modificar Pedido.

**Context**

Pedido

**Strategy**

DescuentoStrategy

**ConcreteStrategy**

SinDescuento
DescuentoEstudiante
DescuentoHechizo

# Problema de diseño en Magic Café

**Pedido sin Strategy**

```
if (tipo.equals("ESTUDIANTE")) {
    total = subtotal * 0.85;
} else if (tipo.equals("HECHIZO")) {
    total = Math.max(0, subtotal - 1000);
} else {
    total = subtotal;
}
```

**Pedido conoce cada descuento**

Las reglas quedan mezcladas con el cálculo del pedido.

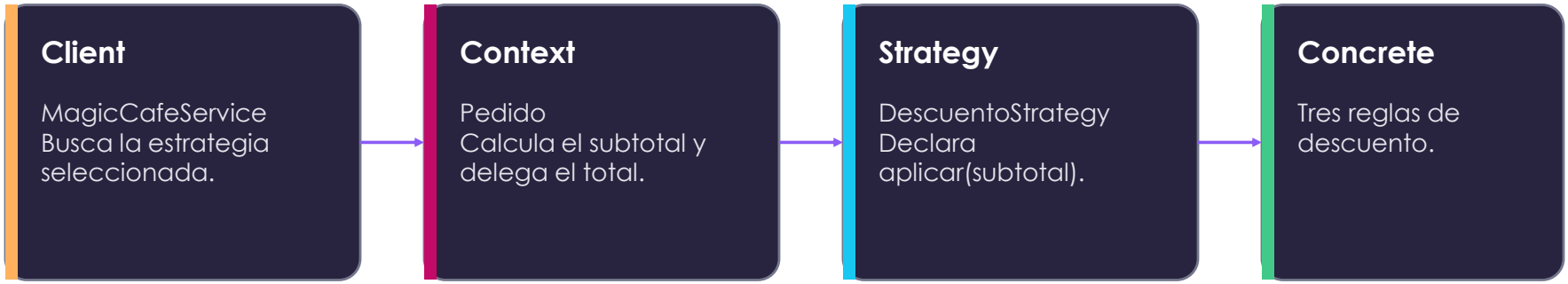
**Agregar un descuento modifica Pedido**

La clase viola Open/Closed a medida que crece el catálogo.

**Las reglas son difíciles de probar**

Cada caso depende del condicional completo.

# Roles del patrón en Magic Café




- La selección ocurre fuera de Pedido. La ejecución ocurre dentro de Pedido.

# Paso 1: definir DescuentoStrategy

**DescuentoStrategy.java**

```
@Entity
@Table(name = "descuentos")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "tipo_descuento")
public abstract class DescuentoStrategy {
    public abstract String getNombre();
    public abstract double aplicar(double subtotal);
}
```

**Contrato común**

Pedido solo necesita aplicar(subtotal).

**Clase abstracta**

Comparte identidad y mapeo JPA entre estrategias.

**La abstracción combina el rol Strategy con una entidad persistente.**

# Paso 2: implementar estrategias concretas

**SinDescuento.java**

```
@Entity
@DiscriminatorValue("SIN_DESCUENTO")
public class SinDescuento
        extends DescuentoStrategy {
    public double aplicar(double subtotal) {
        return subtotal;
    }
}
```

**DescuentoEstudiante.java**

```
@Entity
@DiscriminatorValue("ESTUDIANTE")
public class DescuentoEstudiante
        extends DescuentoStrategy {
    public double aplicar(double subtotal) {
        return subtotal - subtotal * porcentaje /
100;
    }
}
```

- Ambas clases respetan el mismo contrato y encapsulan una sola regla.

# Paso 3: Pedido delega el comportamiento

**Pedido.java**

```
@ManyToOne(cascade = CascadeType.PERSIST,
           fetch = FetchType.EAGER)
@JoinColumn(name = "descuento_id", nullable = false)
private DescuentoStrategy descuento;

public void recalcularTotales() {
    subtotal = items.stream()
            .mapToDouble(ItemPedido::getSubtotal)
            .sum();

    total = descuento.aplicar(subtotal);
}
```

**Responsabilidad de Pedido**

Suma los subtotales de sus items.

**Responsabilidad de Strategy**

Transforma el subtotal en el total final.

# El Service selecciona la estrategia

**MagicCafeService.confirmarPedido(...)**

```
DescuentoStrategy descuento =
        descuentoRepository.findById(descuentoId)
        .orElseThrow(() ->
                new RuntimeException("Descuento no encontrado"));

Pedido pedido = new Pedido(descuento);

for (ItemPedidoRequest item : items) {
    Producto producto = productoRepository
            .findById(item.getProductoId()).orElseThrow();
    pedido.agregarProducto(producto, item.getCantidad());
}

pedidoRepository.save(pedido);
```

**Application**

Orquesta repositorios y crea Pedido.

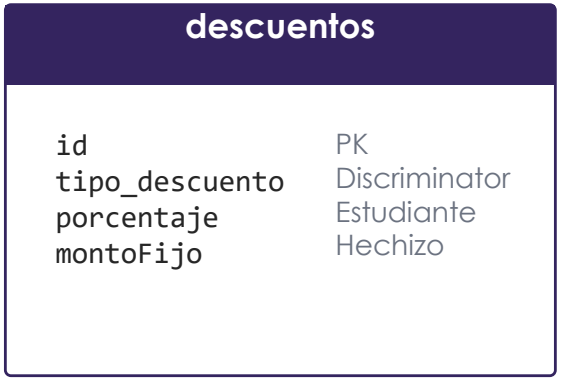
**Domain**

Pedido y la Strategy ejecutan las reglas.

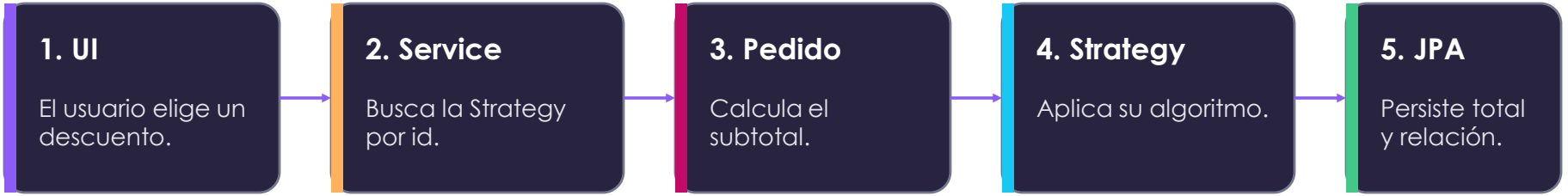
- El Service elige. Pedido delega.

# Strategy persistida con JPA

- DescuentoStrategy usa herencia SINGLE_TABLE.
- tipo_descuento distingue SIN_DESCUENTO, ESTUDIANTE y HECHIZO.
- Pedido mantiene una relación @ManyToOne con la estrategia elegida.
- El catálogo de descuentos se consulta mediante DescuentoRepository.



# Flujo al confirmar un pedido




- Pedido nunca decide qué tipo de descuento recibió.

# Gracias!

- Preguntas?
- mailto: mstricagnoli@uade.edu.ar

